
REQUIREMENTS NOT MET

N/A

VIDEO FILE LINK

<https://youtube.com/shorts/YlnWWnNcygU?si=WH5EoAzmUNSY7Gze>

PROBLEMS ENCOUNTERED

Understanding outputs of my registers, told to focus on output by PI.

FUTURE WORK/APPLICATIONS

With a controller, we can make a CPU and eventually build the backbone of modern computing.

PRE-LAB QUESTIONS OR EXERCISES

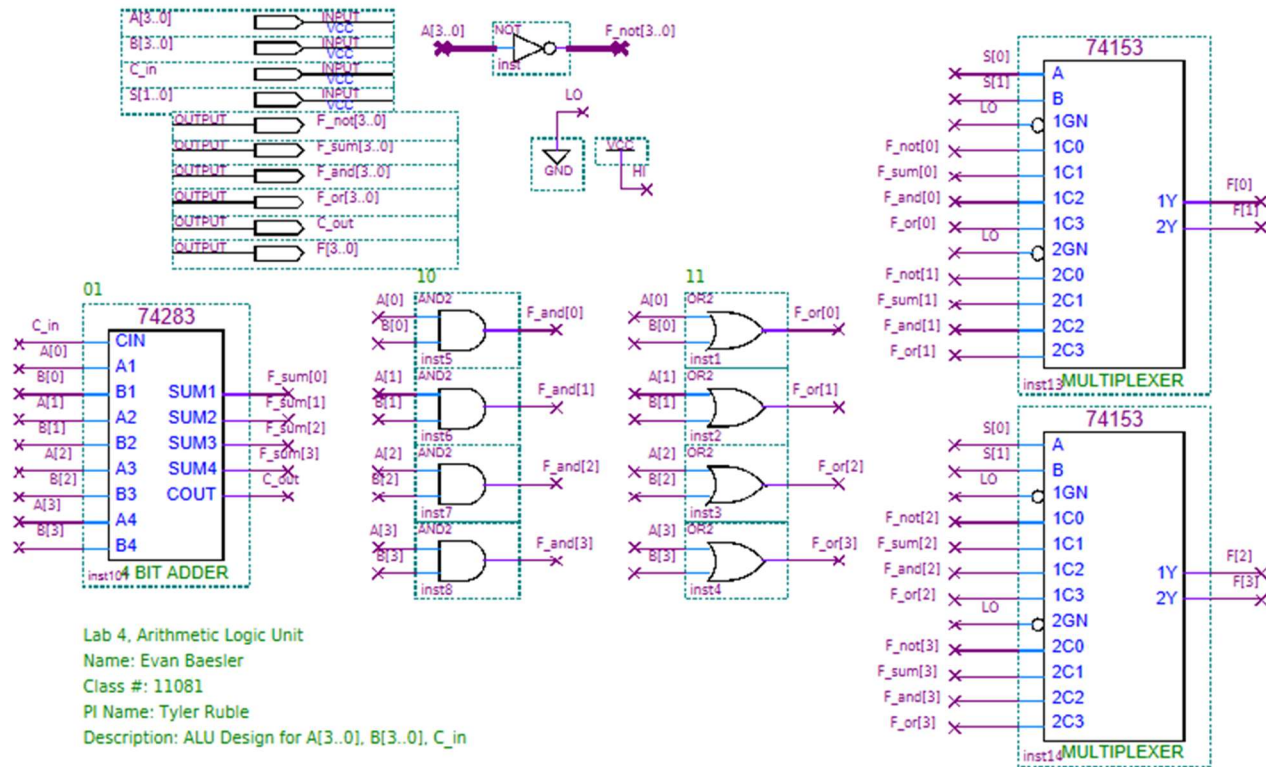
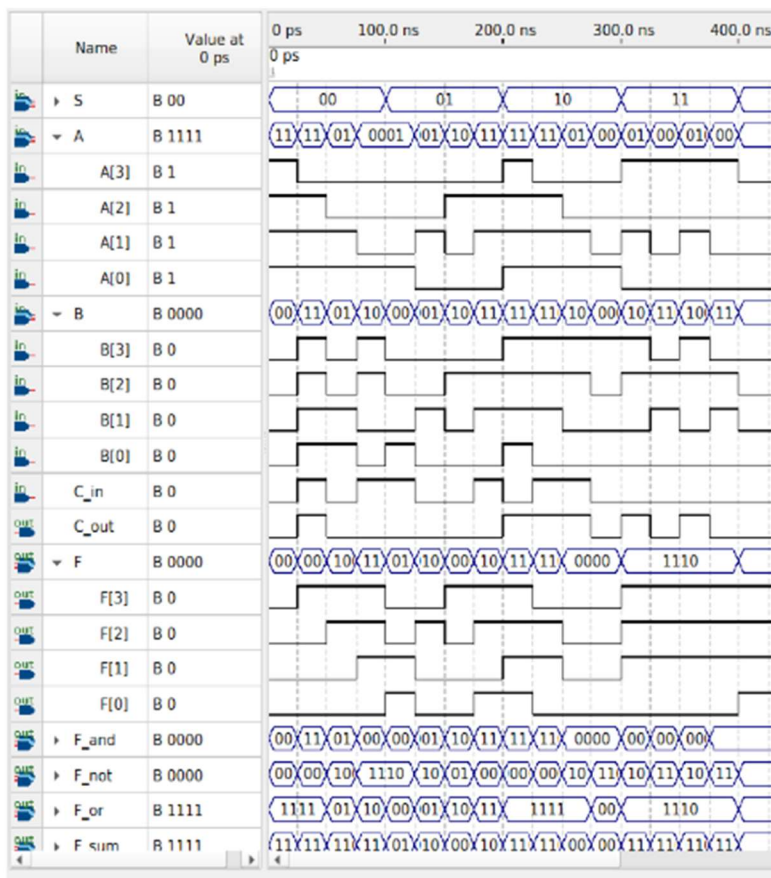


Figure 1: ALU Quartus BDF Design



On Clock 1: Inputs(A,B,Cin) 1111, 0000, X, Function: 00 (COMP A), Out = 0000
On Clock 2: Inputs(A,B,Cin) 0111, 1111, X, Function: 00 (COMP A), Out = 1000
On Clock 3: Inputs(A,B,Cin) 0011, 0011, X, Function: 00 (COMP A), Out = 1100
On Clock 4: Inputs(A,B,Cin) 0001, 1100, X, Function: 00 (COMP A), Out = 1110
On Clock 5: Inputs(A,B,Cin) 0001, 0001, 1, Function: 01 (SUM ABC), Out = 0011
On Clock 6: Inputs(A,B,Cin) 0010, 0010, 0, Function: 01 (SUM ABC), Out = 0100
On Clock 7: Inputs(A,B,Cin) 0100, 0100, 0, Function: 01 (SUM ABC), Out = 1000
On Clock 8: Inputs(A,B,Cin) 0110, 0110, 1, Function: 01 (SUM ABC), Out = 1101
On Clock 9: Inputs(A,B,Cin) 1111, 1111, X, Function: 10 (AND AB), Out = 1111
On Clock 10: Inputs(A,B,Cin) 0111, 1110, X, Function: 10 (AND AB), Out = 0110
On Clock 11: Inputs(A,B,Cin) 0011, 1100, X, Function: 10 (AND AB), Out = 0000
On Clock 12: Inputs(A,B,Cin) 0001, 1000, X, Function: 10 (AND AB), Out = 0000
On Clock 13: Inputs(A,B,Cin) 1010, 1100, X, Function: 11 (OR AB), Out = 1110
On Clock 14: Inputs(A,B,Cin) 1000, 0110, X, Function: 11 (OR AB), Out = 1110
On Clock 15: Inputs(A,B,Cin) 1010, 1100, X, Function: 11 (OR AB), Out = 1110
On Clock 16: Inputs(A,B,Cin) 1000, 0110, X, Function: 11 (OR AB), Out = 1110

Figure 2: Annotated Quartus ALU Waveform

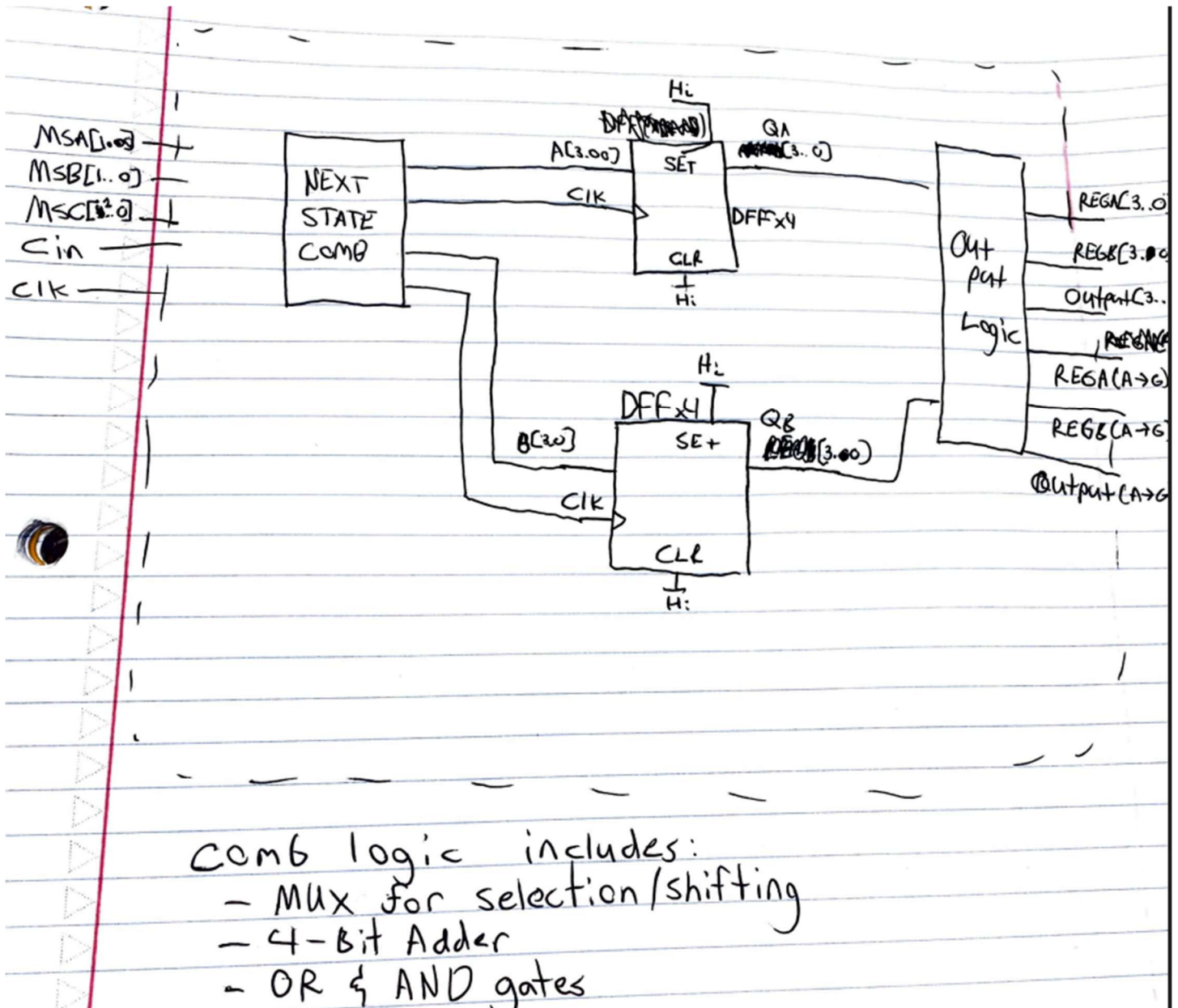


Figure 3: Functional Block Diagram for RALU

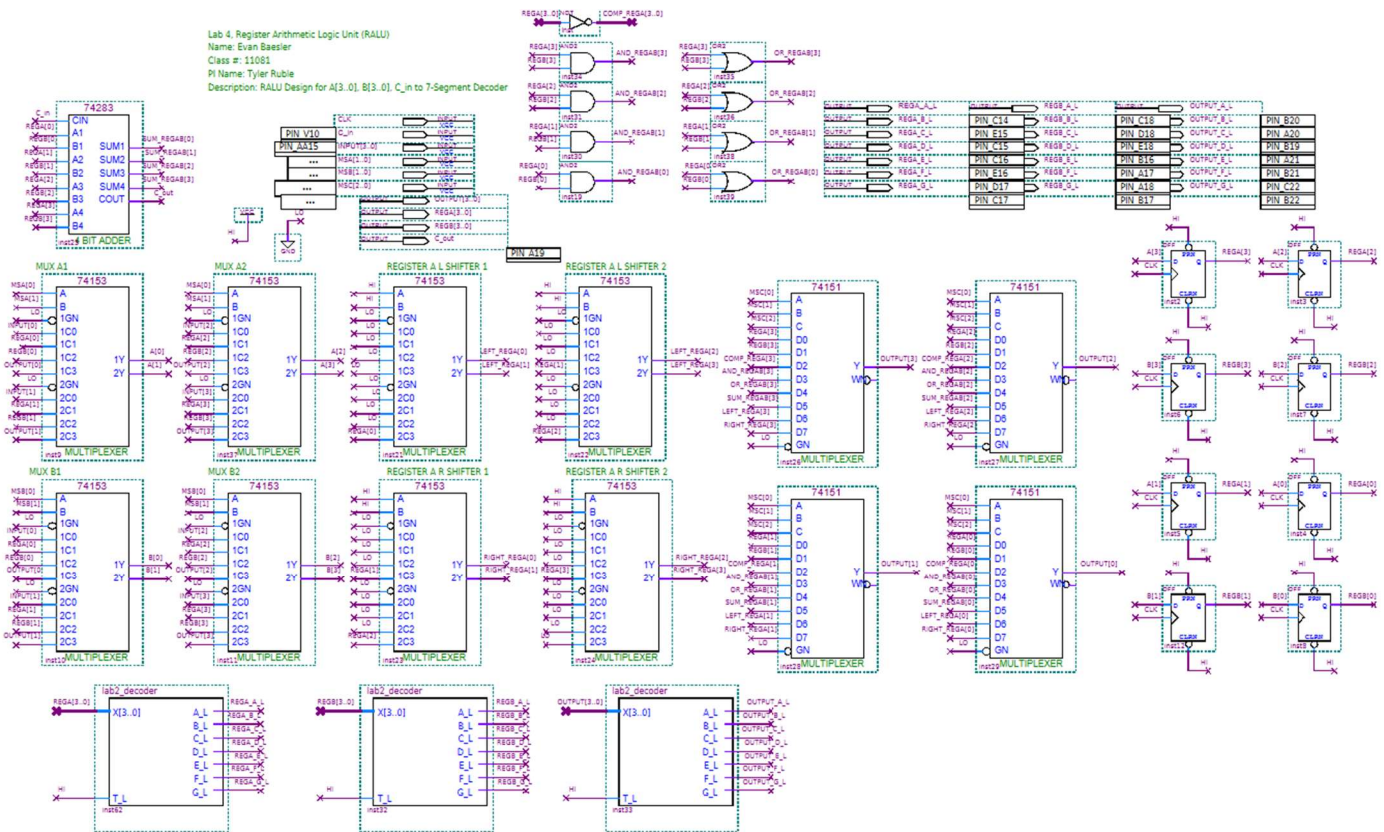
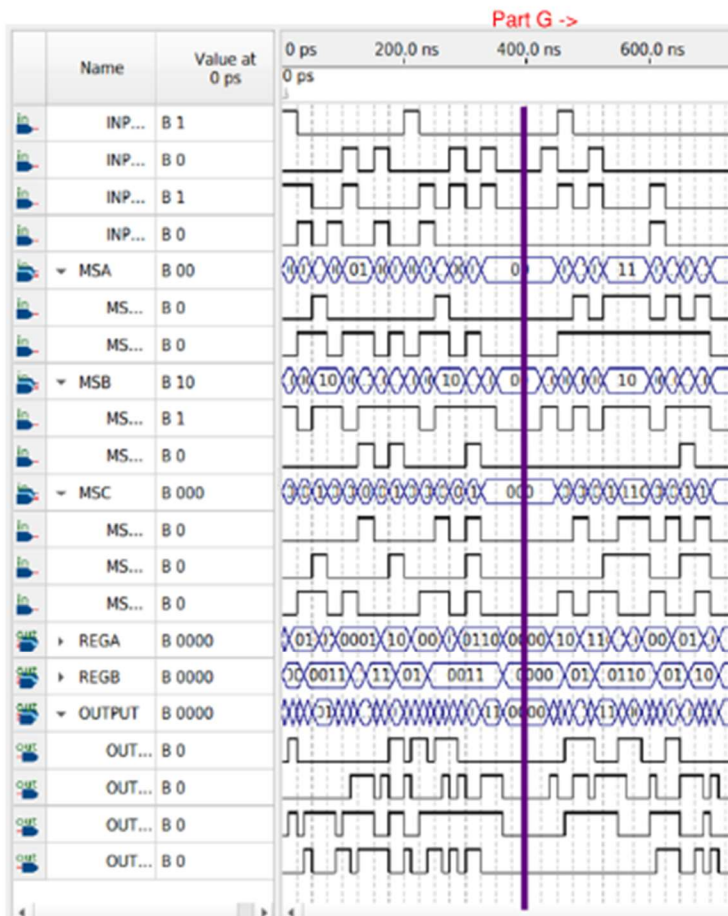


Figure 4: RALU Quartus BDF Design

Table 1: Controls to test all RALU functions. +

MSA	MSB	MSC	Input	Cin	RegA	RegB	Output	RegA+	RegB+	Output+	Cout+	Description
00	10	000	1010	0	X	X	X	1010	X	1010	0	Load 1010 onto REGA
01	00	001	0011	0	1010	X	1010	1010	0011	0011	0	Load 0011 onto RegB
11	10	011	X	0	1010	0011	0010	0010	0011	0010	0	Bit-wise AND, store in REGA
00	10	000	0001	0	0010	0011	0010	0001	0011	0001	0	Load 0001 onto REGA
01	00	001	0110	0	0001	0011	0110	0001	0110	0110	0	Load 0110 onto RegB
01	11	100	X	0	0001	0110	0110	0001	0110	0111	0	Bit-wise OR, store in RegB
00	10	000	0101	0	0001	0110	0111	0101	0110	0101	0	Load 0101 onto REGA
01	11	010	X	0	0101	0110	0101	0101	1010	1010	0	Comp. REGA and store in RegB
00	10	000	1000	0	0101	1010	1010	1000	1010	1000	0	Load 1000 onto REGA
01	00	001	0011	0	1000	1010	1000	1000	0011	0011	0	Load 0011 onto RegB
11	10	101	X	0	1000	0011	0011	1011	0011	1011	0	Sum REGA & REGB and store on REGA
00	10	000	0110	0	1011	0011	1011	0110	0011	0110	0	Load 0110 onto REGA
01	11	111	X	0	0110	0011	0110	0110	0011	0011	0	Shift REGA to the right and store on REGB
00	10	110	0110	0	0110	0011	0011	0110	1100	1100	0	Shift REGA to the left and store on REGB



From the start on clock edge:

On clock 1: Inputs 1010, MSA 00 (in -> regA), MSB 10 (hold), MSC 000 (regA -> out) LOAD REGA
On clock 2: Inputs 0011, MSA 01 (hold), MSB 00 (in -> regB), MSC 001 (regB -> out) LOAD REGB
On clock 3: Inputs 0000, MSA 11 (out -> regA), MSB 10 (hold), MSC 011 (AND regs) BIT-WISE AND
On clock 4: Inputs 0001, MSA 00 (in -> regA), MSB 10 (hold), MSC 000 (regA -> out) LOAD REGA
On clock 5: Inputs 0110, MSA 01 (hold), MSB 00 (in -> regB), MSC 001 (regB -> out) LOAD REGB
On clock 6: Inputs 0000, MSA 01 (hold), MSB 11 (out -> regB), MSC 100 (OR regs), BIT-WISE OR
On clock 7: Inputs 0101, MSA 00 (in -> regA), MSB 10 (hold), MSC 000 (regA -> out) LOAD REGA
On clock 8: Inputs 0000, MSA 01 (hold), MSB 11 (out -> regB), MSC 010 (A Comp) COMPLIMENT
On clock 9: Inputs 1000, MSA 00 (in -> regA), MSB 10 (hold) MSC 000 (regA -> out) LOAD REGA
On clock 10: Inputs 0011, MSA 01 (hold), MSB 00 (in -> regB), MSC 001 (regB -> out) LOAD REGB
On clock 11: Inputs 0000, MSA 11 (out -> regA), MSB 10 (hold), MSC 101 (sum regs) SUM REGAB
On clock 12: Inputs 0110, MSA 01 (in -> regA), MSB 10 (hold) MSC 000 (regA -> out) LOAD REGA
On clock 13: Inputs 0000, MSA 01 (hold), MSB 11 (out -> regB) MSC 111 (SHIFT) RIGHTSHIFT
On clock 14: Inputs 0110, MSA 00 (in -> regA), MSB 10 (hold), MSC 110 (SHIFT) LEFTSHIFT

Part G:

On clock 1: Inputs 0100, MSA 00 (in -> regA), MSB 10 (hold), MSC 000 (regA -> out) LOAD REGA
On clock 2: Inputs 1010, MSA 01 (hold), MSB 00 (in -> regB), MSC 001 (regB -> out) LOAD REGB
On clock 3: Inputs 0000, MSA 11 (out -> regA), MSB 10 (hold), MSC 100, OR REGS
On clock 4: Inputs 0110, MSA 01 (hold), MSB 00 (in -> regB), MSC 001 (regB -> out) LOAD REGB
On clock 5: Inputs 0000, MSA 11 (out -> regA), MSB 10 (hold), MSC 011, BIT-WISE AND
On clock 6: Inputs 0000 MSA 11 (out -> regA), MSB 10 (hold), MSC 110, LEFT SHIFT
On clock 7: Inputs 0000 MSA 11 (out -> regA), MSB 10 (hold), MSC 110, LEFT SHIFT
On clock 8: Inputs 0011, MSA 01 (hold), MSB 00 (in -> regB), MSC 001, LOAD REGB
On clock 9: Inputs 0000, MSA 11 (out -> regA), MSB 10 (hold), MSC 100, BIT-WISE OR
On clock 10: Inputs 0000, MSA 01 (hold), MSB 11 (out -> regB), MSC 010, COMPLIMENT
On clock 11: Inputs 0000, MSA 11 (out -> regA), MSB 10 (hold), MSC 111 SHIFT RIGHT

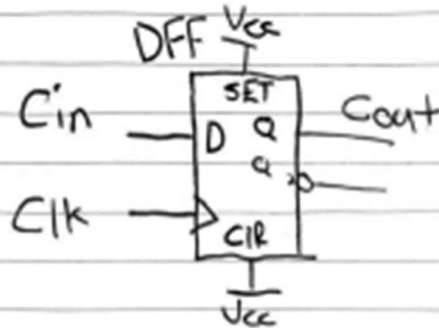
Figure 5: Annotated Quartus RALU Waveform for 4a-g

Table 2: Table for Implementation of Q4g

MSA	MSB	MSC	Input	Cin	RegA	RegB	Output	RegA+	RegB+	Output+	Cout+	Description
00	10	000	0100	0	X	X	X	0100	X	0100	0	Load 4 to REGA
01	00	001	1010	0	0100	X	0100	0100	1010	1010	0	Load \$A to REGB
11	10	100	X	0	0100	1010	1010	1110	1010	1110	0	OR REGA and REGB
01	00	001	0110	0	1110	1010	1110	1110	0110	0110	0	Load 6 to REGB
11	10	011	X	0	1110	0110	1110	0110	0110	0110	0	AND REGA and REGB
11	10	110	X	0	1110	0110	0110	1100	0110	1100	0	Shift Left REGA
11	10	110	X	0	1100	0110	1100	1000	0110	1000	0	Shift Left REGA
01	00	001	0011	0	1000	0110	1000	1000	0011	0011	0	Load 3 to REGB
11	10	100	X	0	1000	0011	0011	1011	0011	1011	0	OR REGA and REGB
01	11	010	X	0	1011	0011	1011	1011	0100	0100	0	Comp. REGA
11	10	111	X	0	1011	0100	0100	0010	0100	0010	0	Shift Right REGA

PRELAB QUESTIONS:

1)



2) ~~NOT~~ Division by 2 is a shift to the right which will only work w/ sign extension

3) ~~More~~ Load A with input
 Complement A
~~Set~~ Load B with 0
 Add $A+B+C_{in}$ for $C_{in} = 1$
 Store in B

4) Load A with negative
~~Load B with positive~~
 Complement A, save to A
 set $B = 0000$
 Add $A+B+C_{in}$ for $C_{in} = 1$, save to A
 Load positive to B
 Add $A+B+C_{in}$ for $C_{in} = 0$

5) Load 0000 to Register/output from input

Figure 6: Q1-5 from Prelab Questions